Intelligent LLM Routing Middleware: Optimizing Cost, Latency, and Performance via a Virtual Meta-Model

Metis-OS Team

September 29, 2026

1 Project Overview

Large Language Models (LLMs) vary drastically in cost, speed, and reasoning capabilities. Currently, users and applications must manually choose between "fast and cheap" models or "slow and expensive" models. This project aims to build a Virtual Meta-Model (e.g., Metis-Auto)—an intelligent middleware layer that automatically dynamically routes user queries to the most appropriate LLM provider. By analyzing the complexity of incoming prompts, the system will optimize for cost and speed without sacrificing response quality.

2 The Problem Statement

To ensure this solution is commercially viable, the architecture must address the pain points that recur throughout the four types of data shipped with this project. The dataset does not distinguish who is using the system; it only records how the system is used, and the design must therefore hold up across every usage pattern it contains:

- **AI Decision Fatigue:** Users suffer from "AI decision fatigue." They shouldn't have to guess whether their question requires a lightweight model or a highly advanced reasoning model. Metis-Auto eliminates the need for manual model switching, automatically delivering instant answers for simple questions and deep reasoning for complex ones, all while draining fewer credits from their account.
- **API Fragmentation:** Building GenAI applications requires juggling multiple APIs, managing rate limits, and implementing fallback logic. A unified, highly available API gateway is needed that handles intelligent routing, semantic caching, and automatic failover under the hood, so that the focus stays on application logic rather than infrastructure.
- **Unit Economics & SLAs:** Operating a chatbot service at scale requires strict control over unit economics and SLA guarantees. The system must

maximize profit margins by deflecting simple queries to cheap models (or caches) while maintaining high user satisfaction, and it must expose observability tools to track token usage, cost savings, and routing decisions in real time.

3 Project Milestones & Technical Requirements

The Metis-Auto engine is built around the following three core actions:

3.1 Action 1: Smart Classifier & Dynamic Routing

Design and implement a lightweight, low-latency classifier to evaluate incoming queries.

- **Feature Extraction:** Analyze the prompt for language, semantic intent, code presence, and multimodal attachments (image/audio).
- **Routing Logic:** Route simple, conversational, or highly structured inputs to fast, economical models (e.g., Gemini 1.5 Flash, GPT-4o-mini). Route complex, multi-step reasoning, or highly technical queries to advanced models (e.g., Claude 3.5 Sonnet, o3-mini).
- *Constraint:* The classifier must be computationally cheap so it does not add significant latency or cost to the pipeline.

3.2 Action 2: Cascade Architecture & Semantic Caching

Implement an optimization and reliability layer on the gateway.

- **Semantic Cache:** Build a vector-based semantic cache to identify and instantly serve answers to repetitive or conceptually identical queries, entirely bypassing LLM API costs.
- **Escalation & Failover:** Design a "Cascade" protocol. If a cheaper model fails to generate a satisfactory response, the system should seamlessly escalate the query to a stronger model. If a provider API goes down, the system must automatically failover to a parallel provider to ensure 100

3.3 Action 3 [Optional]: Financial Modeling & Observability Dashboard

Bridge the gap between machine learning and product economics.

- **Pricing Engine:** Design a financial model (fixed subscription vs. floating dynamic pricing) that proves Metis-Auto is more economical for the end-user than querying premium models directly.
- **Transparency Dashboard:** Develop a user-facing observability panel that visualizes the "value created." It must display metrics such as tokens saved via caching, cost reductions from intelligent routing, and overall system latency.

4 Dataset & Evaluation

The project includes a real-world dataset, `sample-data/data.json`, containing 100 anonymized chatbot sessions (907 messages in total) collected from production traffic between January and September 2026. The dataset is divided into four equally sized usage categories (25 sessions each). These categories are intentionally left unnamed in this document to encourage solutions that generalize across usage patterns rather than being tuned to any specific one. The full-scale dataset will be provided later.

Each session includes an identifier, a creation timestamp, the number of user and bot turns, and the complete ordered transcript. Every message contains a role (`user` or `bot`), content, and a timestamp. Bot messages additionally include the model and provider that generated the response (18 distinct models across 7 providers) and, when retrieval was used, the retrieved context. In the current sample, a minority of messages include attachments (14 messages) or structured input widgets (29 messages), and 71 messages contain retrieval context. The content is predominantly in Persian, making language detection a core part of the task rather than an afterthought. All personally identifiable information has been removed from both the messages and the retrieved contexts.

4.1 Evaluation

- **Quality:** average semantic similarity (and, where feasible, LLM-as-a-judge scores) of routed vs. actual bot answers;
- **Semantic-cache hit rate:** fraction of queries served from the cache;
- **Cost:** simulated cost per query vs. the single premium-model baseline;
- **Latency:** end-to-end response time vs. the baseline.